

MODULE 09

Dimensionality Reduction and PCA

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Dimensionality Reduction and PCA

Why Reduce Dimensions?

High-dimensional data causes:

Curse of dimensionality: more features = harder to learn

Overfitting: model memorises noise

Slow training: computational cost

Visualization impossible beyond 3D

Reduction benefits:

Faster training, less overfitting, easier visualization, remove noise

Principal Component Analysis PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Always scale first!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Fit PCA
pca = PCA()
pca.fit(X_scaled)

# Explained variance
import matplotlib.pyplot as plt
plt.plot(range(1, len(pca.explained_variance_ratio_)+1),
pca.explained_variance_ratio_.cumsum())
plt.xlabel('Number of components')
plt.ylabel('Cumulative variance explained')
plt.axhline(y=0.95, color='r', linestyle='--')
```

Apply PCA

```
from sklearn.decomposition import PCA

# Keep 95% of variance
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_scaled)

print(f'Original features: {X_scaled.shape[1]}')
print(f'Reduced features: {X_reduced.shape[1]}')

# Use in pipeline
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
pipe = Pipeline([
```

```

('scaler', StandardScaler()),
('pca', PCA(n_components=0.95)),
('clf', LogisticRegression())
])
pipe.fit(X_train, y_train)

```

t-SNE for Visualisation

```

from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# t-SNE for 2D visualization (not for training!)
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_2d = tsne.fit_transform(X_scaled)

plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='tab10')
plt.colorbar()
plt.title('t-SNE Visualization')
plt.show()

# UMAP (faster than t-SNE)
from umap import UMAP
umap = UMAP(n_components=2)
X_2d = umap.fit_transform(X_scaled)

```

Feature Selection

```

from sklearn.feature_selection import (
    SelectKBest, f_classif, RFE, mutual_info_classif
)
from sklearn.ensemble import RandomForestClassifier

# Statistical test
selector = SelectKBest(f_classif, k=10)
X_selected = selector.fit_transform(X, y)

# Recursive Feature Elimination
rfe = RFE(RandomForestClassifier(), n_features_to_select=10)
X_rfe = rfe.fit_transform(X, y)

# Feature importance
rf = RandomForestClassifier()
rf.fit(X, y)
importances = pd.Series(rf.feature_importances_, index=feature_names)

```